Carrera: Ingeniería en Sistemas Computacionales.

Asignatura: Métodos Cuantitativos aplicados a las finanzas para la toma de decisiones.

Docente: I.S.C Juan Reynoso Elías.

Alumno: Alejandro Vasquez Nuñez.

Fecha De Elaboracion: 29/10/2026.

Semestre: 7 Semestre.

Tema: Manual de examen de Métodos Cuantitativos aplicados a las finanzas para la toma de decisiones.

Salon: 3711

1) **Primer Ejercicio.**
Punto de equilibrio? Nivel de ventas en el que los ingresos totales de un negocio son iguales a sus costos totales (Fijos / Variables) lo que significa que la empresa no genera ni pérdidas ni ganancias.

Ejemplo del 1 ejercicio.

Planteamiento del ejemplo

Una tienda de mochilas vende cada mochila a un precio de **$50 USD**. El costo de fabricación de cada mochila es de **$20 USD**, se paga una comisión de ventas de **$5 USD** por unidad, y los gastos fijos mensuales ascienden a **$5,000 USD**. ¿Cuál es el punto de equilibrio en unidades?

Pasos para realizar el ejercicio:

1) **Identificar los costos variables unitarios:** Sumar el costo de producción o compra más la comisión de ventas por unidad.

Costo Variable Unitario = costo de produccion + Comision

Costo Variable Unitario = 20 + 5 = 25 usd

2) Calcular el margen de contribución unitario: Restar el costo variable unitario al precio de venta. Esto representa cuánto aporta cada unidad vendida para cubrir los costos fijos

Margen de contribucion = Precio de venta - Costo Variable unitario

Margen de contribucion = 50 - 25 = 25 usd

3) Calcular el punto de equilibrio en unidades: Dividir los costos fijos totales entre el margen de contribución unitario.

Punto de equilibrio = Costo Fijos / Margen de contribucion

Punto de equilibrio = 5000/25 = 200 unidades

1) Ejemplo codigo relacionado al ejemplo resolvido

```
# Datos del ejemplo de las mochilas
precio_venta_ej = 50
costo_produccion_ej = 20
 comision_ventas_ej = 5
costos_fijos_ej = 5000
# 1. Costo variable unitario total
costo_variable_ej = costo_produccion_ej + comision_ventas_ej
# 2. Margen de contribución
margen_contribucion_ej = precio_venta_ej - costo_variable_ej
# 3. Punto de equilibrio en unidades
punto_equilibrio_ej = costos_fijos_ej / margen_contribucion_ej
print("--- Resultado del Ejemplo (Mochilas) ---")
print(f"Costo variable unitario: ${costo_variable_ej} USD")
print(f"Margen de contribución: ${margen_contribucion_ej} USD")
print(f"Punto de equilibrio: {punto_equilibrio_ej} unidades")
```

Definición de Variables Iniciales: Se asignan los valores numéricos del problema a variables con nombres descriptivos. En este caso: cada mochila se vende a $50 USD, cuesta $20 USD producirla, paga $5 USD de comisión por unidad, y los costos fijos mensuales son de $5,000 USD.

Cálculo del Costo Variable Unitario: Suma el costo de producción por unidad y la comisión de ventas (20 + 5 = 25). Esto representa todo el dinero que le cuesta a la empresa fabricar y vender cada mochila adicional.

Cálculo del Margen de Contribución Unitario: Resta el costo variable total al precio de venta (50 - 25 = 25). Este resultado indica cuánto dinero aporta cada mochila vendida para ayudar a pagar los gastos fijos de la empresa.

Cálculo del Punto de Equilibrio en Unidades: Divide los costos fijos totales entre el margen de contribución (5000 / 25). El resultado obtenido es 200, lo que significa que la tienda necesita vender exactamente **200 mochilas** para cubrir todos sus gastos sin generar pérdidas ni ganancias.

Impresión de Resultados (print): Utiliza cadenas formateadas (f-strings, indicadas por la letra f antes de las comillas) para mostrar en la pantalla de la consola de Python los resultados de cada operación matemática de forma clara y ordenada junto con sus respectivas unidades (USD y unidades).

1) **Problema / Codigo del primer ejercicio de examen**

```
precio_venta = 40
costo_camisa = 24
comision = 2
costos_fijos = 3500
costo_variable_total = costo_camisa + comision
margen_contribucion = precio_venta - costo_variable_total
punto_equilibrio_unidades = costos_fijos / margen_contribucion


print("=== Solución del Problema Original (Camisas) ===")
print(f"Costo Variable Unitario: ${costo_variable_total} USD")
print(f"Margen de Contribución por unidad: ${margen_contribucion} USD")
print(f"Punto de Equilibrio: {punto_equilibrio_unidades} unidades de camisas")
```

Almacenamiento de variables: Se guardan los datos iniciales del problema en variables numéricas (precio_venta, costo_camisa, comision, costos_fijos).

Costo variable unitario (costo_variable_total): Agrupa todos los costos que cambian dependiendo de cuántas camisas se produzcan y vendan ($24 + 2 = 26$).

Margen de contribución (margen_contribucion): Muestra el dinero libre que deja cada camisa vendida después de descontar sus costos directos y comisiones ($40 - 26 = 14$).

Cosultor del punto de equilibrio (punto_equilibrio_unidades): Divide los gastos fijos totales entre lo que aporta cada camisa ($3,500 / 14$). Da como resultado **250 unidades**, lo que significa que la empresa necesita vender exactamente 250 camisas para no tener ganancias ni pérdidas (cubrir todos sus costos).

"Una empresa dedicada a la comercialización de camisas, vende camisas a un precio de $40 USD, el costo de cada camisa es de$ 24 USD, se paga una comisión de ventas por $2 USD, y sus gastos fijos (alquiler, salarios, servicios, etc.), ascienden a$ 3,500 USD. ¿Cuál es el punto de equilibrio en unidades de venta?." **250 Unidades de camisas**

Indicación adicional: Realizarlo en una hoja de cálculo y en un programa en python.

Punto_de_Equilibrio_Ejercicio_1.xlsx

B	C	D
CÁLCULO DEL PUNTO DE EQUILIBRIO		
Concepto	**Valor**	**Unidad / Notas**
Precio de venta por unidad	$40,00	USD
Costo de cada camisa (Producción)	$24,00	USD
Comisión de ventas por unidad	$2,00	USD
Gastos Fijos Totales	$3.500,00	USD
Cálculos e Indicadores	**Fórmula / Resultado**	**Unidad**
Costo Variable Unitario Total	$26,00	USD
Margen de Contribución Unitario	$14,00	USD
Punto de Equilibrio en Unidades	**250**	Unidades de camisas
Punto de Equilibrio en Ventas Totales	**$10.000,00**	USD

2) Segundo Ejercicio

Ejemplo Similar y Pasos de Resolución

Supongamos que tenemos un archivo llamado tipo_cambio_ejemplo.xls que contiene una columna llamada "TipoCambio" con una serie de valores diarios del dólar. Queremos calcular sus métricas estadísticas clave.

Pasos para realizar el análisis estadístico

1) **Cargar los datos:** Leer el archivo de Excel usando una librería especializada (como pandas en Python o readxl en R).
2) **Limpiar y preparar los datos:** Seleccionar la columna numérica de interés y asegurar que no existan valores vacíos (NaN).
3) **Calcular la Media (Promedio):** Sumar todos los valores y dividirlos entre el número total de datos.
4) **Calcular la Varianza y Desviación Estándar:** Medir qué tan dispersos están los datos respecto a la media.
5) **Obtener el Máximo y Mínimo:** Encontrar el valor más alto y más bajo de la serie.
6) **Calcular el Rango:** Restar el valor mínimo al valor máximo (Maximo / Minimo)

1) Ejemplo codigo relacionado Python

```
import pandas as pd
# 1. Cargar el archivo de Excel de ejemplo
df_ejemplo = pd.read_excel("tipo_cambio_ejemplo.xls")
# 2. Seleccionar la columna de tipo de cambio (asumimos que se llama 'TipoCambio')
datos_ej = df_ejemplo["TipoCambio"].dropna()
# 3. Calcular las métricas solicitadas
media_ej = datos_ej.mean()
varianza_ej = datos_ej.var()                    # Varianza muestral
desviacion_ej = datos_ej.std()                  # Desviación estándar muestral
maximo_ej = datos_ej.max()
minimo_ej = datos_ej.min()
rango_ej = maximo_ej - minimo_ej

# 4. Mostrar resultados
print("--- Métricas del Ejemplo en Python ---")
print(f"Media: {media_ej:.4f}")
print(f"Varianza: {varianza_ej:.4f}")
print(f"Desviación Estándar: {desviacion_ej:.4f}")
print(f"Máximo: {maximo_ej:.4f}")
print(f"Mínimo: {minimo_ej:.4f}")
print(f"Rango: {rango_ej:.4f}")
```

Explicación Detallada Línea por Línea:

1) **Importar la librería Pandas (import pandas as pd):**Se importa **Pandas**, que es la biblioteca principal en Python para el manejo, lectura y análisis de estructuras de datos tabulares (como archivos de Excel o CSV). Se le asigna el alias pd para escribir código más limpio y corto.

2) Cargar el archivo de Excel (pd.read_excel):

Python

df_ejemplo = pd.read_excel("tipo_cambio_ejemplo.xls")

La función read_excel abre el archivo especificado (tipo_cambio_ejemplo.xls) y lo convierte en un **DataFrame** (una tabla virtual en la memoria de Python con filas y columnas).

3) Seleccionar la columna y limpiar los datos (["TipoCambio"].dropna()):

Python

datos_ej = df_ejemplo["TipoCambio"].dropna()

df_ejemplo["TipoCambio"] extrae únicamente la columna que contiene los valores numéricos del tipo de cambio.

.dropna() es una función de limpieza muy importante que elimina automáticamente cualquier celda vacía (NaN o nula) para evitar que ocurran errores matemáticos al momento de calcular las estadísticas.

4) Cálculo de la Media o Promedio (.mean()):

media_ej = datos_ej.mean()

Suma todos los valores de la columna y los divide entre el número total de registros para obtener el promedio aritmético de todo el periodo.

5) Cálculo de la Varianza (.var()):

varianza_ej = datos_ej.var()

Calcula la **varianza muestral**, que mide qué tandispersos o alejados están los datos individuales con respecto al promedio central.

6) Cálculo de la Desviación Estándar (.std()):

desviacion_ej = datos_ej.std()

Calcula la **desviación estándar muestral**. Es la raíz cuadrada de la varianza y sirve para saber, en promedio, qué tanto fluctúa el tipo de cambio respecto a la media en las mismas unidades originales.

7) Obtener el Valor Máximo y Mínimo (.max() y .min()):

maximo_ej = datos_ej.max() minimo_ej = datos_ej.min()

Buscan de manera automática dentro de toda la serie de datos el valor numérico más alto alcanzado por el dólar y el valor más bajo, respectivamente.

8) Cálculo del Rango (maximo_ej - minimo_ej):

rango_ej = maximo_ej - minimo_ej

Realiza una resta simple entre el valor máximo y el valor mínimo. Esto indica la amplitud total de variación que experimentó el dólar entre su punto más alto y su punto más bajo.

Código de Ejemplo en el Lenguaje R

```
# Instalar y cargar la librería para leer archivos de Excel (si no está instalada: install.packages("readxl"))
library(readxl)
# 1. Cargar el archivo de Excel de ejemplo
df_ejemplo <- read_excel("tipo_cambio_ejemplo.xls")

# 2. Seleccionar la columna de tipo de cambio
datos_ej <- na.omit(df_ejemplo$TipoCambio)

# 3. Calcular las métricas estadísticas
media_ej <- mean(datos_ej)
varianza_ej <- var(datos_ej)
esviacion_ej <- sd(datos_ej)
maximo_ej <- max(datos_ej)
minimo_ej <- min(datos_ej)
rango_ej <- maximo_ej - minimo_ej

# 4. Mostrar resultados en consola
cat("--- Métricas del Ejemplo en R ---\n")
cat("Media:", round(media_ej, 4), "\n")
cat("Varianza:", round(varianza_ej, 4), "\n")
cat("Desviación Estándar:", round(desviacion_ej, 4), "\n") cat("Máximo:", round(maximo_ej, 4), "\n")
cat("Mínimo:", round(minimo_ej, 4), "\n")
cat("Rango:", round(rango_ej, 4), "\n")
```

Explicación Detallada Línea por Línea:

1. Cargar la librería (library(readxl))

library(readxl): En R, para leer archivos en formato Excel (.xls o .xlsx), se utiliza el paquete o librería readxl. Esta línea activa la librería para poder usar sus funciones. (Si no está instalada, se instala previamente con install.packages("readxl")).

2. Lectura del archivo (read_excel) y Asignación (<-)

df_ejemplo <- read_excel("tipo_cambio_ejemplo.xls"):

El operador <- es el símbolo nativo de asignación en R (equivalente al signo = en otros lenguajes).

La función read_excel() lee el archivo y lo almacena en un Data Frame (tabla de datos) llamado df_ejemplo.

3. Extracción de columna y limpieza ($ y na.omit())

df_ejemplo$TipoCambio: En R, el operador $ sirve para acceder directamente a una columna específica dentro de un Data Frame.

na.omit(...): Elimina automáticamente cualquier valor ausente o celda vacía (NA en R, que significa *Not Available*). Los datos limpios se guardan en el vector datos_ej.

4. Funciones Estadísticas Nativas

R es un lenguaje diseñado especialmente para estadística, por lo que sus funciones principales vienen integradas de forma directa:

mean(datos_ej): Calcula el promedio o media aritmética.

· sd(datos_ej): Calcula la desviación estándar muestral que representa la raíz cuadrada de la varianza.

· · max(datos_ej) y min(datos_ej): Obtienen el valor máximo y mínimo registrado de la serie histórica.

· · rango_ej <- maximo_ej - minimo_ej: Realiza la resta directa para obtener el rango o amplitud total de la variación del tipo de cambio.

5. Impresión de Resultados (cat() y round())

- cat(...): Es la función estándar en R para imprimir texto y variables de manera corrida y formateada en la consola (a diferencia de print(), cat() concatena varios elementos fácilmente).

· · round(variable, 4): Redondea el valor numérico a **4 decimales**, asegurando que los resultados se presenten con la misma precisión que solicita la tabla de tu evaluación.

· · "\n": Representa un salto de línea para que los resultados aparezcan uno debajo del otro.

Primer Codigo Python

```
import pandas as pd
def calcular_metricas(nombre_archivo):
df = pd.read_excel(nombre_archivo, header=None)
      for col in df.columns:
serie = pd.to_numeric(df[col], errors='coerce')
if serie.dropna().count() > 5:
datos = serie.dropna()
Break
else:
datos = pd.to_numeric(df.iloc[:, 1], errors='coerce').dropna()
metricas = {
"Media": datos.mean(),
"Varianza": datos.var(),
"Desviación estándar": datos.std(),
"Máximo": datos.max(),
"Mínimo": datos.min(),
"Rango": datos.max() - datos.min()
}
return pd.Series(metricas)
print("=== RESULTADOS EN PYTHON ===")
print("\n--- Sexenio 1 ---")
print(calcular_metricas("/home/juice/Documentos/Examen_Metodos/sexenio-1.xls"))
print("\n--- Sexenio 2 ---")
print(calcular_metricas("/home/juice/Documentos/Examen_Metodos/sexenio-2.xls"))
```

Explicacion del codigo

1) Carga la biblioteca **Pandas**, encargada de la lectura y procesamiento de los archivos de Excel.
2) Se crea una función llamada calcular_metricas que recibe como parámetro la ruta del archivo (nombre_archivo) para que pueda reutilizarse tanto para el Sexenio 1 como para el Sexenio 2.

pd.read_excel(..., header=None) abre el archivo de Excel indicándole que **no asuma que la primera fila son los nombres de las columnas**, sino que lea todo como datos tabulares limpios.

3) · **Bucle for:** Recorre columna por columna de izquierda a derecha en busca de aquella que contenga los datos numéricos reales del tipo de cambio.

· · pd.to_numeric(..., errors='coerce'): Intenta convertir el texto de la celda a número; si encuentra letras, títulos o celdas vacías, los convierte en valores nulos (NaN) sin que el programa falle.

· · if serie.dropna().count() > 5: Revisa si la columna tiene más de 5 números válidos. Si los encuentra, los selecciona y detiene la búsqueda (break).

· · **Bloque else:** Si por alguna razón el bucle no encuentra la columna automáticamente, asigna por defecto la **columna de índice 1** (la segunda columna del archivo)

4- ● Se crea un diccionario de Python llamado metricas que agrupa los cálculos matemáticos utilizando las funciones integradas de Pandas:

datos.mean(): Calcula el promedio de todo el periodo.

datos.var(): Obtiene la varianza.

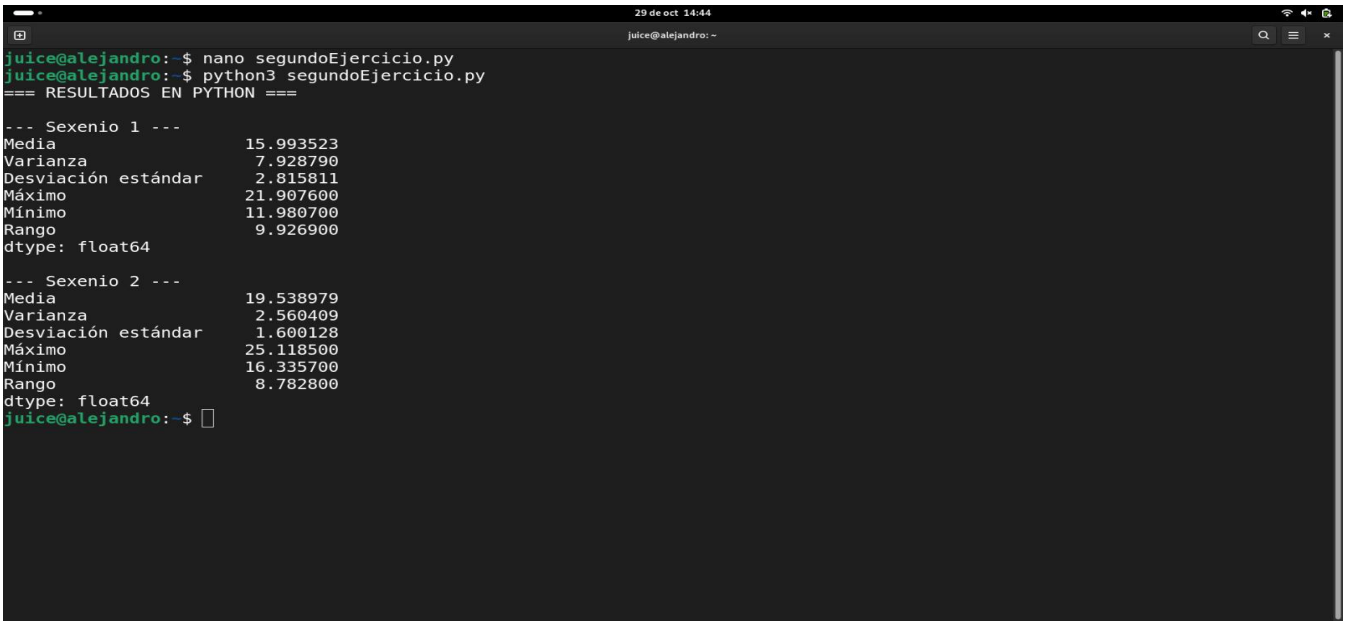
datos.std(): Calcula la desviación estándar.

datos.max() y datos.min(): Extraen el valor más alto y más bajo.

datos.max() - datos.min(): Resta el máximo menos el mínimo para obtener el rango.

return pd.Series(metricas): Convierte el diccionario en una serie de datos ordenada para que se imprima de forma tabular clara.

5- Manda a llamar la función pasándole las rutas exactas de tus archivos en la computadora (sexenio-1.xls y sexenio-2.xls) e imprime en la consola los resultados finales de cada métrica solicitada en tu evaluación.



Resultados del primer codigo realizado en lenguaje de programacion en **Python**

Primer Codigo R

```
library(readxl)
df <- read_excel(ruta_archivo, col_names = FALSE)
datos <- NULL
for (col in df) {
num_col <- suppressWarnings(as.numeric(as.character(col)))
if (sum(!is.na(num_col)) > 5) {
datos <- num_col[!is.na(num_col)]
Break
}
}
media <- mean(datos)
varianza <- var(datos)
desv_est <- sd(datos)
maximo <- max(datos)
minimo <- min(datos)
rango <- maximo - minimo

resultado <- c(
"Media" = media,
"Varianza" = varianza,
"Desviación estándar" = desv_est,
"Máximo" = maximo,
"Mínimo" = minimo,
"Rango" = rango
)
return(resultado)
}

cat("=== RESULTADOS EN R ===\n")
cat("\n--- Sexenio 1 ---\n")
print(calcular_metricas("/home/juice/Documentos/Examen_Metodos/sexenio-1.xls"))
cat("\n--- Sexenio 2 ---\n")
print(calcular_metricas("/home/juice/Documentos/Examen_Metodos/sexenio-2.xls"))
```

Explicación Bloque por Bloque:

1. Cargar la librería para leer Excel: Activa el paquete readxl, el cual es necesario en R para poder importar y leer archivos con formato de Excel (.xls o .xlsx).

Definición de la función calcular_metricas: · Se crea una función reutilizable llamada calcular_metricas que recibe como entrada la ruta de cada archivo de sexenio.

read_excel(..., col_names = FALSE) abre el archivo de Excel indicándole a R que **no asuma que la primera fila tiene los títulos de las columnas**, sino que lea todo el contenido como datos puros desde la primera celda.

3. Búsqueda automática de la columna numérica: ·**Bucle for:** Recorre cada columna del archivo de Excel de manera automática.

as.character(col) y as.numeric(...): Intentan transformar los datos a números. Si encuentran letras, textos o celdas vacías, R genera valores faltantes (NA). La función suppressWarnings oculta las advertencias por esos textos que no se pudieron convertir.

· · sum(!is.na(num_col)) > 5: Cuenta cuántos números válidos reales existen en esa columna. Si encuentra más de 5, significa que esa es la columna con la información económica buscada, la guarda en la variable datos y detiene la búsqueda (break).

4. Cálculo de las métricas estadísticas

· mean(datos): Calcula la media o promedio aritmético del tipo de cambio.

· · var(datos): Calcula la varianza muestral.

· · sd(datos): Calcula la desviación estándar muestral.

· · max(datos) y min(datos): Obtienen el valor máximo y mínimo registrado en todo el periodo.

· · rango <- maximo - minimo: Resta el valor máximo menos el mínimo para obtener la amplitud total de la fluctuación.

5. Empaquetar y retornar el resultado

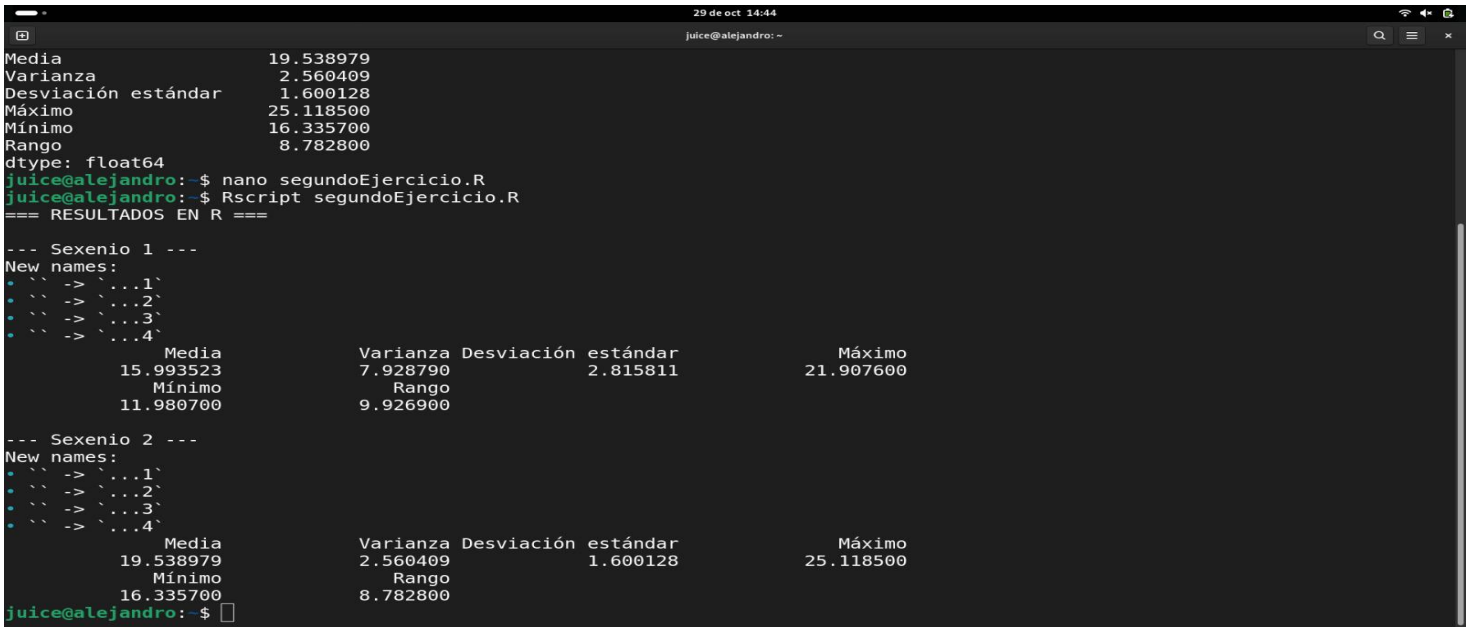
· La función c(...) agrupa todos los valores calculados en un **vector con nombre** (similar a una lista con etiquetas descriptivas).

· · return(resultado) devuelve estos datos ya procesados cada vez que se manda a llamar la función.

6. Ejecución del programa para ambos sexenios

· **cat(...)**: Imprime en la consola textos organizados y títulos con saltos de línea (\n).

· · **print(calcular_metricas(...))**: Llama a nuestra función pasando las rutas exactas de los archivos de cada sexenio y muestra en pantalla la tabla con todas las métricas estadísticas calculadas automáticamente



Resultados del codigo realizado en lenguaje de programacion en **R.**

Tercer ejercicio

Planteamiento del ejemplo

Supongamos que queremos predecir el precio de alquiler de un departamento basándonos únicamente en su tamaño en **metros cuadrados**. Tenemos una tabla con datos históricos de precios y metros cuadrados.

Pasos para realizar un modelo de regresión lineal en R:

Definir las variables: Identificar la variable independiente (predictora, ej. metros cuadrados X) y la variable dependiente (a predecir, ej. precio Y).

Crear el modelo de regresión: Utilizar la función lineal de R lm(Y ~ X) para ajustar la línea recta que mejor se adapte a los datos (mínimos cuadrados).

Obtener los coeficientes: Extraer la pendiente y la intersección con el eje Y

Visualizar o graficar el modelo: Opcionalmente, trazar los puntos y la línea de tendencia.

```
# 1. Crear datos de ejemplo
metros_cuadrados <- c(50, 60, 70, 80, 90, 100)
precio_alquiler <- c(500, 580, 650, 720, 810, 890)

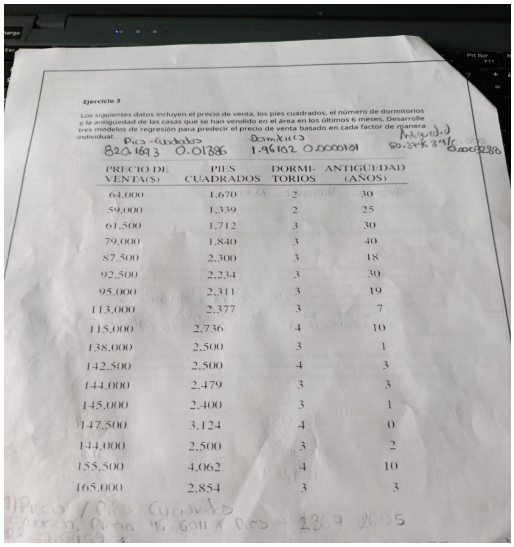
# 2. Crear el modelo de regresión lineal (Precio en función de los metros)
modelo_ejemplo <- lm(precio_alquiler ~ metros_cuadrados)

# 3. Ver los coeficientes del modelo (Intersección y Pendiente)
print("--- Coeficientes del Ejemplo ---")
print(coef(modelo_ejemplo))

# 4. Resumen estadístico completo del modelo
summary(modelo_ejemplo)
```

Un pequeño ejemplo relacionado al ejercicio que tenemos que es el tres a lo cual nos demuestra como relacionarlo en lenguaje de programacion y poder solucionarlo.

Problema / solucion de codigo



Ejercicio 3

Los siguientes datos incluyen el precio de venta, los pies cuadrados, el número de dormitorios y la antigüedad de las casas que se han vendido en el área en los últimos 6 meses. Desarrolle tres modelos de regresión para predecir el precio de venta basado en cada factor de manera individual.

PRECIO DE VENTA(S)	PIES CUADRADOS	DORMI-TORIOS	ANTIGÜEDAD (AÑOS)
64,000	1,670	2	30
59,000	1,339	2	25
61,500	1,712	3	30
79,000	1,840	3	40
87,500	2,300	3	18
92,500	2,234	3	30
95,000	2,311	3	19
113,000	2,377	3	7
115,000	2,736	4	10
138,000	2,500	3	1
142,500	2,500	4	3
144,000	2,479	3	3
145,000	2,400	3	1
147,500	3,124	4	0
144,000	2,500	3	2
155,500	4,062	4	10
165,000	2,854	3	3

Los siguientes datos incluyen el precio de venta, los pies cuadrados, el número de dormitorios y la antigüedad de las casas que se han vendido en el área en los últimos 6 meses. Desarrolle tres modelos de regresión para predecir el precio de venta basado en cada factor de manera individual.?

Pies Cuadrados = 820.1693 / 0.01386

Dormitorios = 1.96102 - 0.0000101

Antiguedad = 50.374684 / 0.0003288

```
import pandas as pd
import numpy as np
datos = {
'Precio': [64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000, 115000, 138000, 142500,
144000, 145000, 147500, 144000, 155500, 165000],
Pies_Cuadrados': [1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500, 2500, 2479, 2400,
3124, 2500, 4062, 2854],
'Dormitorios': [2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3],
'Antiguedad': [30, 25, 30, 40, 18, 39, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10, 3]
}
df = pd.DataFrame(datos)
y = df['Precio']
variables = ['Pies_Cuadrados', 'Dormitorios', 'Antiguedad']
print("=== MODELOS DE REGRESIÓN LINEAL SIMPLE ===\n")
for var in variables:
x = df[var]
m, b = np.polyfit(x, y, 1)
correlacion = np.corrcoef(x, y)[0, 1]
r2 = correlacion**2
print(f"Modelo para: {var}")
print(f" Ecuación: Precio = {m:.4f} * ({var}) + {b:.4f}")
```

```
print(f" R² (Coef. de Determinación): {r2:.4f}")
print("-" * 50)
```

Explicación Bloque por Bloque:

1. Importar las librerías necesarias

- **Pandas (pd):** Se utiliza para estructurar y organizar los datos originales de la tabla en un formato tabular limpio (DataFrame).

NumPy (np): Es la librería matemática fundamental en Python; la usamos aquí para

hacer los cálculos vectoriales rápidos, las regresiones polinómicas y las correlaciones

estadísticas.

2. Cargar los datos de la tabla

· Se crea un diccionario llamado datos que almacena de manera idéntica los valores numéricos de las 17 casas que aparecen en tu evaluación.

· · df = pd.DataFrame(datos) convierte ese diccionario en una tabla virtual.

· · y = df['Precio'] separa la columna del precio de venta, la cual funcionará como la **variable dependiente** (Y, lo que queremos predecir).

· · variables guarda una lista con los nombres de los tres factores independientes que evaluaremos de forma individual: los pies cuadrados, el número de dormitorios y la antigüedad.

Bucle para procesar cada modelo de manera individual

· El bucle for recorre uno por uno los elementos de la lista variables.

· · En cada iteración, x = df[var] extrae únicamente la columna correspondiente al factor que se está analizando en ese momento.

4. Cálculo de la pendiente y la ordenada al origen por mínimos cuadrados.

· np.polyfit(x, y, 1) ajusta una línea recta (polinomio de grado 1) utilizando el método de **mínimos cuadrados**.

· · Devuelve dos valores clave:·

m (Pendiente): Cuánto cambia el precio de la casa por cada unidad que aumenta el factor.

b (Ordenada al origen / Intercepto): El valor base del precio cuando la variable independiente es cero.

5. Cálculo del coeficiente de correlación y R^2

· np.corrcoef(x, y) calcula la matriz de correlación de Pearson entre la variable y el precio . Al extraer [0, 1] obtenemos el valor exacto de la correlación lineal.

· · r2 = correlacion**2 calcula el **coeficiente de determinación (R^2)**, elevando al cuadrado la correlación. Este indicador porcentual nos dice qué tan bien se ajusta el modelo y qué porcentaje de la variación del precio de las casas es explicado por ese factor en específico.

5. Impresión y formato de resultados

Utiliza plantillas de texto formateadas (f-strings) para imprimir en la pantalla de la consola la ecuación de regresión resultante de cada modelo con sus decimales exactos (.4f), facilitando la lectura directa de los resultados de tu tarea.

```
29 de oct 14:47
juice@alejandro: ~
  GNU nano 8.4                                  tercerEjercicio.R *
S## Datos de la tabla del Ejercicio 3
precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000, 115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500, 165000)
pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500, 2500, 2479, 2400, 3124, 2500, 4062, 2854)
dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)
antiguedad <- c(30, 25, 30, 40, 18, 39, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10, 3)

df <- data.frame(Precio = precio, Pies = pies, Dormitorios = dormitorios, Antiguedad = antiguedad)

cat("=== MODELOS DE REGRESIÓN LINEAL SIMPLE EN R ===\n\n")

# 1. Pies Cuadrados
modelo_pies <- lm(Precio ~ Pies, data = df)
s_pies <- summary(modelo_pies)
cat("Modelo: Precio vs Pies Cuadrados\n")
cat(sprintf("  Ecuación: Precio = %.4f * Pies + %.4f\n", coef(modelo_pies)[2], coef(modelo_pies)[1]))
cat(sprintf("  R²: %.4f\n", s_pies$r.squared))
cat("------------------------------------------------\n")

# 2. Dormitorios
modelo_dorm <- lm(Precio ~ Dormitorios, data = df)
s_dorm <- summary(modelo_dorm)
cat("Modelo: Precio vs Dormitorios\n")
cat(sprintf("  Ecuación: Precio = %.4f * Dormitorios + %.4f\n", coef(modelo_dorm)[2], coef(modelo_dorm)[1]))
cat(sprintf("  R²: %.4f\n", s_dorm$r.squared))
cat("------------------------------------------------\n")

# 3. Antigüedad
modelo_ant <- lm(Precio ~ Antiguedad, data = df)
s_ant <- summary(modelo_ant)
cat("Modelo: Precio vs Antigüedad\n")
cat(sprintf("  Ecuación: Precio = %.4f * Antiguedad + %.4f\n", coef(modelo_ant)[2], coef(modelo_ant)[1]))
cat(sprintf("  R²: %.4f\n", s_ant$r.squared))
cat("------------------------------------------------\n")

^G Ayuda      ^O Guardar     ^F Buscar      ^K Cortar      ^T Ejecutar    ^C Ubicación   M-U Deshacer   M-A Poner marca  M-] A llave
^X Salir      ^R Leer fich.  ^\ Reemplazar  ^U Pegar       ^J Justificar  ^/ Ir a línea  M-E Rehacer    M-6 Copiar       ^B Buscar atrás
```

1) Demostracion del codigo como se hizo.

```
juice@alejandro: $ python3 tercerEjercicio.py
=== MODELOS DE REGRESIÓN LINEAL SIMPLE ===

Modelo para: Pies_Cuadrados
  Ecuación: Precio = 46.6011 * (Pies_Cuadrados) + 2367.2605
  R² (Coef. de Determinación): 0.6457
------------------------------------------------
Modelo para: Dormitorios
  Ecuación: Precio = 36137.7551 * (Dormitorios) + 1923.4694
  R² (Coef. de Determinación): 0.3651
------------------------------------------------
Modelo para: Antiguedad
  Ecuación: Precio = -2198.2333 * (Antiguedad) + 145751.4248
  R² (Coef. de Determinación): 0.7256
```

2) Resultados del ejercicio numero tres.

Cuarto Ejercicio

1. Ejemplo Similar y Pasos de Resolución

Planteamiento del ejemplo

Supongamos que queremos predecir el precio de renta de oficinas utilizando dos factores simultáneamente: los **metros cuadrados** de la oficina y el **número de privados (habitaciones)**.

Pasos para realizar un modelo de regresión lineal múltiple en R:

Definir las variables:

Variable dependiente : Lo que queremos predecir (ej. Precio).

Variables independientes: Los factores predictores (ej. Metros cuadrados y Número de privados).

Crear el modelo de regresión múltiple: Utilizar la función lineal múltiple en R lm(Y ~ X1 + X2), la cual ajusta un plano de estimación en lugar de una línea recta.

Hacer una predicción: Sustituir los valores específicos solicitados en la ecuación resultante para obtener el resultado estimado.

Ejemplo del codigo

```
# 1. Crear datos de ejemplo
metros_cuadrados <- c(40, 50, 60, 70, 80)
privados <- c(1, 1, 2, 2, 3)
precio_alquiler <- c(600, 750, 900, 1050, 1200)
# 2. Crear el modelo de regresión múltiple.
modelo_ejemplo <- lm(precio_alquiler ~ metros_cuadrados + privados)
# 3. Ver los coeficientes del modelo
print("--- Coeficientes del Ejemplo Múltiple ---")
print(coef(modelo_ejemplo))
# 4. Predecir para una oficina con 75 metros cuadrados y 2 privados
nueva_oficina <- data.frame(metros_cuadrados = 75, privados = 2)
prediccion_ej <- predict(modelo_ejemplo, nueva_oficina)
cat("Precio estimado de la oficina:", prediccion_ej, "\n")
```

Explicación Paso a Paso de su Funcionamiento:

Creación de vectores de datos independientes y dependientes:

· metros_cuadrados <- c(...) y privados <- c(...): Representan las **variables independientes (X_1 y X_2)**, es decir, los factores que influyen en el precio.

· · precio_alquiler <- c(...): Representa la **variable dependiente (Y)**, que es el precio de alquiler que deseamos explicar y predecir.

Ajuste del Modelo de Regresión Múltiple (lm):

· modelo_ejemplo <- lm(precio_alquiler ~ metros_cuadrados + privados)

· · La función lm() (*linear model*) calcula de manera matemática (mediante mínimos cuadrados matriciales) un plano de estimación. El símbolo ~ significa "en función de", y el signo + indica que estamos sumando dos variables independientes para predecir el resultado de forma simultánea.

Extracción de Coeficientes (coef):

```
print(coef(modelo_ejemplo))
```

Muestra la intersección (constante o valor base) y las pendientes asociadas a cada variable. Estos valores conforman la ecuación matemática de estimación.

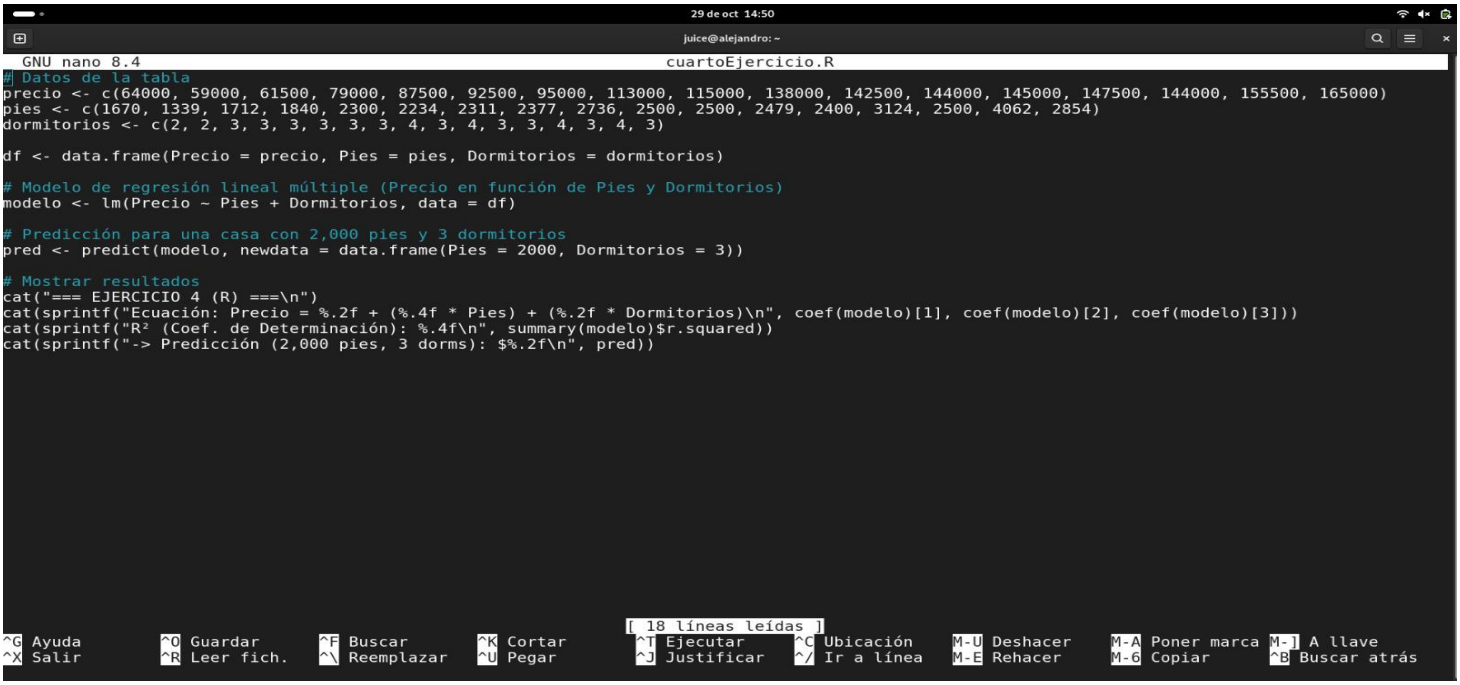
Predicción con Nuevos Datos (predict):

· nueva_oficina <- data.frame(metros_cuadrados = 75, privados = 2): Crea una tabla con los datos específicos de un caso nuevo que queremos calcular.

· · prediccion_ej <- predict(modelo_ejemplo, nueva_oficina): Sustituye automáticamente esos valores nuevos en la ecuación generada por el modelo para arrojar el resultado estimado de alquiler.

· · cat(...): Imprime en la consola el resultado final formateado de manera clara.

Explicacion de codigo del problema original




```
# Datos de la tabla
precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000, 115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500, 165000)
pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500, 2500, 2479, 2400, 3124, 2500, 4062, 2854)
dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)


df <- data.frame(Precio = precio, Pies = pies, Dormitorios = dormitorios)
```

· **c(...)**: Crea vectores de datos con la información numérica de las 17 casas recopiladas en la tabla de la evaluación.

· · **precio**: Variable dependiente (Y, el precio de venta en USD).

· · **pies y dormitorios**: Variables independientes (X_1 y X_2, los pies cuadrados y la cantidad de dormitorios de cada casa).

· · **data.frame(...)**: Junta los tres vectores en una sola tabla organizada con nombres de columnas amigables (Precio, Pies, Dormitorios).

```
# Modelo de regresión lineal múltiple (Precio en función de Pies y Dormitorios)
modelo <- lm(Precio ~ Pies + Dormitorios, data = df)
```

· **lm(...)** (*linear model*): Ajusta un modelo de regresión lineal múltiple por el método de mínimos cuadrados ordinarios.

· · **Precio ~ Pies + Dormitorios**: La virgulilla (~) indica que estamos prediciendo el Precio basándonos en la combinación de Pies más Dormitorios de forma simultánea.

```
# Predicción para una casa con 2,000 pies y 3 dormitorios
pred <- predict(modelo, newdata = data.frame(Pies = 2000, Dormitorios = 3))
```

data.frame(Pies = 2000, Dormitorios = 3): Define las características de la casa que nos pide calcular el ejercicio.

```
# Mostrar resultados
cat("=== EJERCICIO 4 (R) ===\n")
cat(sprintf("Ecuación: Precio = %.2f + (%.4f * Pies) + (%.2f * Dormitorios)\n", coef(modelo)[1], coef(modelo)[2], coef(modelo)[3]))
cat(sprintf("R² (Coef. de Determinación): %.4f\n", summary(modelo)$r.squared))
cat(sprintf("-> Predicción (2,000 pies, 3 dorms): $%.2f\n", pred))
```

summary(modelo)$r.squared: Extrae el valor del coeficiente de determinación (R^2), el cual indica qué porcentaje de la variación del precio de las casas es explicado por ambas variables.

sprintf(...): Formatea los números con precisión especificada (%.2f para dos decimales y %.4f para cuatro decimales) dentro de las cadenas de texto antes de imprimirlas en la consola con cat(...).

```
=== EJERCICIO 4 (R) ===
Ecuación: Precio = 5701.45 + (48.5054 * Pies) + (-2540.39 * Dormitorios)
R² (Coef. de Determinación): 0.6465
-> Predicción (2,000 pies, 3 dorms): $95091.18
juice@alejandro: $
```

Resultados del ejercicio cuartro

Quinto ejercicio

te presento la explicación detallada, un ejemplo similar, el paso a paso de su código en R, y la resolución completa con el código en R para el Ejercicio 5 de tu evaluación (Regresión Lineal Múltiple utilizando tres variables independientes: pies cuadrados, número de dormitorios y antigüedad de la casa

1. Ejemplo Similar y Pasos de Resolución

Planteamiento del ejemplo

Supongamos que queremos predecir el precio de alquiler de locales comerciales utilizando tres factores simultáneamente: los metros cuadrados, el número de privados, y los años de antigüedad del edificio.

Pasos para realizar un modelo de regresión múltiple con tres variables en R:

Definir las variables:

Variable dependiente (Y): Lo que queremos predecir (ej. Precio).

Variables independientes (X_1, X_2, X_3): Los tres factores predictores.

Crear el modelo de regresión múltiple: Utilizar la función lm(Y ~ X1 + X2 + X3) en R para ajustar un hiperplano de estimación con todas las variables explicativas.

Hacer una predicción: Sustituir los valores específicos solicitados en la ecuación resultante para obtener el valor estimado final.

```
# 1. Crear datos de ejemplo con 3 variables independientes
metros <- c(40, 50, 60, 70, 80)
privados <- c(1, 1, 2, 2, 3)
antiguedad_ej <- c(5, 10, 2, 15, 1)
precio_alq <- c(600, 750, 900, 1050, 1200)
# 2. Crear el modelo de regresión múltiple
 modelo_ej <- lm(precio_alq ~ metros + privados + antiguedad_ej)
# 3. Ver los coeficientes
print("--- Coeficientes del Ejemplo de Regresión Múltiple ---") print(coef(modelo_ej))
# 4. Predecir para un local con 65 metros, 2 privados y 5 años de antigüedad
nuevo_local <- data.frame(metros = 65, privados = 2, antiguedad_ej = 5)
pred_ej <- predict(modelo_ej, nuevo_local) cat("Precio estimado del local:", pred_ej, "\n")
```

Explicación Paso a Paso de su Funcionamiento:

Creación de los Vectores de Datos:

metros <- c(...), privados <- c(...) y antiguedad_ej <- c(...): Representan las **variables independientes (X_1, X_2, X_3)**, es decir, los tres factores diferentes que se analizan de manera conjunta para influir en el precio.

precio_alq <- c(...): Representa la **variable dependiente (Y)**, que es el precio de alquiler que deseamos explicar y predecir.

Ajuste del Modelo de Regresión Múltiple (lm):

```
modelo_ej <- lm(precio_alq ~ metros + privados + antiguedad_ej)
```

La función lm() (*linear model*) calcula matemáticamente mediante mínimos cuadrados un **hiperplano de estimación**. El símbolo ~ indica que el precio está "en función de" las tres variables independientes conectadas por el signo +.

Extracción de Coeficientes (coef):

```
print(coef(modelo_ej))
```

Muestra en la consola la intersección (constante base) y cada una de las tres pendientes individuales asociadas a los metros, los privados y la antigüedad. Esto conforma la ecuación matemática del modelo

Predicción con Nuevos Datos (predict):

nuevo_local <- data.frame(metros = 65, privados = 2, antiguedad_ej = 5): Crea una tabla temporal con las características específicas de un local nuevo que queremos calcular.

pred_ej <- predict(modelo_ej, nuevo_local): Sustituye automáticamente esos valores específicos en la ecuación generada por el modelo para arrojar el valor monetario estimado.

cat(...): Imprime de forma clara el resultado final en la pantalla de la consola.

Ejercicio Resuelto del examen.

```
29 de oct 14:51
juice@alejandro: ~
  GNU nano 8.4                                  quintoEjercicio.R
# Datos de la tabla (Precio, Pies, Dormitorios, Antigüedad)
precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000, 115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500, 165000)
pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500, 2500, 2479, 2400, 3124, 2500, 4062, 2854)
dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)
antiguedad <- c(30, 25, 30, 40, 18, 39, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10, 3)

df <- data.frame(Precio = precio, Pies = pies, Dormitorios = dormitorios, Antiguedad = antiguedad)

# Modelo de regresión lineal múltiple con las tres variables independientes
modelo <- lm(Precio ~ Pies + Dormitorios + Antiguedad, data = df)

# Predicción solicitada: casa de 2,000 pies, 3 dormitorios y 10 años de antigüedad
pred <- predict(modelo, newdata = data.frame(Pies = 2000, Dormitorios = 3, Antiguedad = 10))

# Mostrar resultados
cat("=== EJERCICIO 5 (R) ===\n")
cat(sprintf("Ecuación: Precio = %.2f + (%.4f * Pies) + (%.2f * Dormitorios) + (%.2f * Antigüedad)\n",
            coef(modelo)[1], coef(modelo)[2], coef(modelo)[3], coef(modelo)[4]))
cat(sprintf("R² (Coef. de Determinación): %.4f\n", summary(modelo)$r.squared))
cat(sprintf("-> Predicción (2,000 pies, 3 dorms, 10 años): $%.2f\n", pred))
                                [ 20 líneas leídas ]
^G Ayuda      ^O Guardar     ^F Buscar      ^K Cortar      ^T Ejecutar    ^C Ubicación   M-U Deshacer   M-A Poner marca  M-] A llave
^X Salir      ^R Leer fich.  ^\ Reemplazar  ^U Pegar      ^J Justificar  ^/ Ir a línea  M-E Rehacer    M-6 Copiar       ^B Buscar atrás
```

1) **Vectores de datos:** Guarda las 4 columnas de la tabla original (precio, pies, dormitorios, antiguedad) en vectores numéricos.

2) · **data.frame(...):** Agrupa todas las columnas en una sola tabla llamada df.
3) · · **lm(...):** Crea la regresión lineal múltiple con las tres variables independientes a la vez (Precio ~ Pies + Dormitorios + Antiguedad).
4) · · **predict(...):** Sustituye los valores pedidos (2,000 pies, 3 dormitorios, 10 años) en la ecuación matemática obtenida para estimar el precio final.
5) · · **cat() y sprintf():** Imprime los resultados en pantalla bien formateados: la ecuación, el valor de R^2 y el precio estimado en dólares.

```
=== EJERCICIO 5 (R) ===
Ecuación: Precio = 74087.62 + (28.3678 * Pies) + (-2126.42 * Dormitorios) + (-1494.24 * Antigüedad)
R² (Coef. de Determinación): 0.8648
-> Predicción (2,000 pies, 3 dorms, 10 años): $109501.63
juice@alejandro:~$
```

Resultados del quinto ejercicio